

StreamingVLA: Streaming Vision-Language-Action Model with Action Flow Matching and Adaptive Early Observation

Yiran Shi¹, Dongqi Guo¹, Tianchen Zhao¹, FengGao¹, Liangzhi Shi¹, Chao Yu¹, ZhiJian Mo², Qihua Xiao², XiaoShuai Peng², Qingmin Liao¹, and Yu Wang¹

¹ Tsinghua University

² Lenovo Group Ltd.

Abstract. Vision-language-action (VLA) models have demonstrated exceptional performance in natural language-driven perception and control. However, the high computational cost of VLA models poses significant efficiency challenges, particularly for resource-constrained edge platforms in real-world deployments. However, since different stages of VLA (observation, action generation and execution) must proceed sequentially, and wait for the completion of the preceding stage, the system suffers from frequent halting and high latency. To address this, We conduct a systematic analysis to identify the challenges for fast and fluent generation, and propose enabling VLAs with the ability to asynchronously parallelize across VLA stages in a "**streaming**" manner. First, we eliminate the reliance on action chunking and adopt action flow matching, which learns the trajectory of action flows rather than denoising chunk-wise actions. It overlaps the latency of action generation and execution. Second, we design an action saliency-aware adaptive observation mechanism, thereby overlapping the latency of execution and observation. Without sacrificing performance, StreamingVLA achieves substantial speedup and improves the fluency of execution. It achieves a $2.4 \times$ latency speedup and reduces execution halting by $6.5 \times$.

1 Introduction

Embodied systems require strong generalization capabilities to operate effectively across diverse real-world scenarios. Inspired by the recent success of foundation models, such as large language models (LLMs) [18, 23] and vision-language models (VLMs) [26], that can perform a wide range of multimodal tasks, **vision-language-action (VLA)** [9, 11, 22, 28, 29, 42, 43] models have emerged as a promising direction toward building foundation models for embodied agents. Despite their impressive performance, the computational cost of large-scale VLA models poses significant efficiency challenges [14, 16], especially on edge platforms. With the adoption of the action chunking technique [15, 24, 41], which

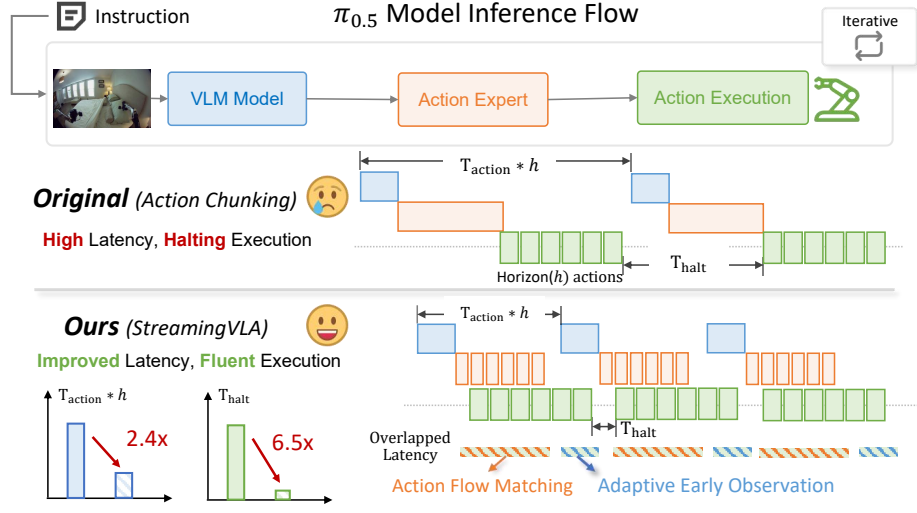


Fig. 1: StreamingVLA explores streaming execution of VLA models by enabling different stage to be run in an asynchronous manner. By overlapping the latency for stages, without sacrificing performance, it achieves $2.4\times$ end-to-end speedup, and $6.5\times$ execution halting time reduction, enabling fast and fluent execution.

generates chunks of actions in parallel, the latency challenge is alleviated. However, there is still room for further improvement [34, 37].

Aside from the overall high latency challenge, current VLA execution also suffers from **inconsecutive and stuttered execution** [21, 32]. Taking the $\pi_{0.5}$ model as an example, it integrates a pre-trained vision-language model (VLM) for observation processing and a diffusion-based action expert for action generation. As illustrated in the timeline diagram in Fig. 1, the multi-stage model inference and action execution proceed in a **synchronous** manner, where each stage must wait for the other to complete. Consequently, a significant waiting period where the system stalls, denoted as T_{halt} , occurs between successive executions, leading to noticeable halting.

A line of research [6, 35] explores parallelizing execution with VLM-based observation processing to address the aforementioned issue. To further improve performance and gain a clearer understanding of the latency and stalling challenges in VLA execution, as well as the potential of **asynchronous and parallelized “streaming” execution across stages**, we conduct a systematic runtime analysis of multi-stage VLAs (Sec. 3). Based on this analysis, we first explore a new dimension of parallelized execution across action generation and execution. We introduce **state-based modeling of action flow matching** (Sec. 4.1), which reformulates action modeling: instead of learning an absolute value mapped from noise, the prediction updates a state conditioned on prior actions, better reflecting the action’s physical meaning. This formulation removes the reliance on action chunking and enables each action to be executed

immediately upon generation. Secondly, to further overlap action execution with the next iteration’s observation processing, we identify action saliency, which describes the extent to which an action influences subsequent observations. We find that action saliency varies significantly across actions, and existing techniques that treat each action uniformly may yield suboptimal performance preservation. Accordingly, we design an **action-saliency-aware adaptive early observation scheme** (Sec. 4.2). We summarize our contribution as follows:

- We perform systematic runtime analysis of the VLA model inference and execution, and conclude optimization targets for fast and fluent execution.
- We present the StreamingVLA framework, which enables asynchronous and parallelized processing of multiple stages of VLA system.
- Without sacrificing performance, StreamingVLA achieves $2.4\times$ speedup, and reduce the halting time by $6.5\times$ to the minimum, enabling fast and fluent execution.

2 Related work

Vision-Language-Action Models: Vision-language-action (VLA) models emerge as a promising direction toward building foundation models for embodied action generation. Early advances, such as RT-1 [8], introduced a model capable of fundamental visual-semantic comprehension for robotic control. RT-2 [7] further extended this paradigm by incorporating vision-language modeling component, enhancing reasoning and generalization. π_0 [4,17] model series integrated a pre-trained VLM with a diffusion-based policy, for environment understanding and action generation. It achieves remarkable performance, and attracted research interest. In this work, we apply our techniques to the $\pi_{0.5}$ model to validate its effectiveness.

Model Compression for VLA: A line of existing research focuses on improving the efficiency of VLAs by adapting model compression techniques. Prior literature [10,40] applies pruning and quantization to reduce the cost of VLM model, while other studies [20,31,38,39] focus on token compression or part correction to reduce data to be processed. Although substantially reduce the latency of each stage for VLA, relying solely on them faces challenges in minimizing action execution haltings (as discussed in Sec. 3). We adopt a different perspective, which overlaps the latency for different stages. It could work in parallel with existing techniques and minimize halting to achieve fluent execution.

Parallelized Execution for VLA: Action chunking [2,25] has been widely adopted, which improves execution efficiency by generating multiple actions in parallel for each observation. RTC [5,6], VLASH [35] and SmolVLA [33] introduce early observation mechanism through finetuning or test-time adjustment that performs VLM midway through ongoing action execution, which enables overlapping action execution and model inference. To enhance the early observation with the ability to distinguish diverse action saliency, we introduce an adaptive observation scheme (Sec. 4.2). Beyond early observation, StreamingVLA provides an extensive runtime analysis (Sec. 3) and extends parallelized

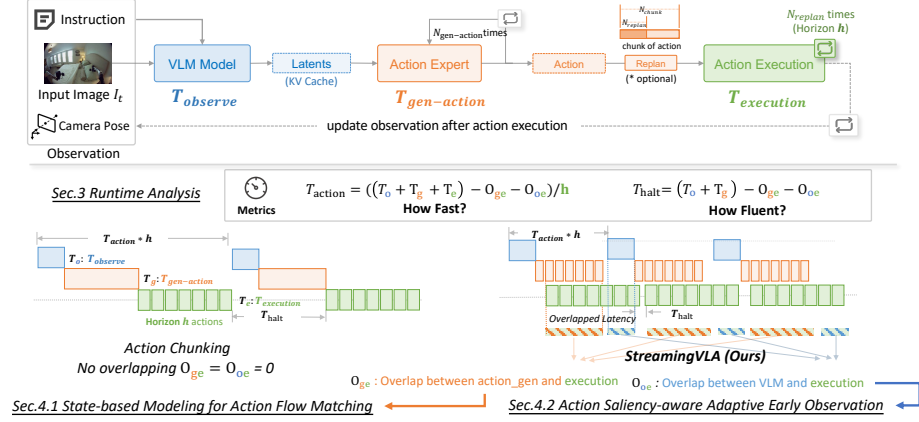


Fig. 2: The overall methodology framework for StreamingVLA: We conduct a systematic timeline analysis and conclude the optimization target for fast and fluent execution. We present two key techniques: action flow matching and adaptive early observation, to overlap the latency of action execution with of action generation and VLM observation, respectively.

execution to a new dimension by enabling stage-wise parallelism between action generation and execution (Sec. 4.1) with state-based action flow matching inspired by SFP [19].

3 Preliminary Analysis of VLA Runtime

To gain a deeper understanding of the issue of VLA’s inconsecutive and slow execution, and the potential of asynchronous and parallelized execution across stages. We conduct a systematic **runtime analysis**. As shown in the upper part of Fig. 2, we illustrate the overall workflow of the VLA model’s inference and action execution, along with the corresponding timeline diagrams. The workflow represents VLAs composed of VLM and diffusion expert (such as π_0 series [4, 17], Gr00t [3], GR3 [1]), we use $\pi_{0.5}$ model as the example, but the analysis is generally applicable. The process can be divided into three main stages: First, the “observation” stage, where the VLM generates latents (kv-cache) based on the current observation, which includes the camera image, language instruction, and the robot arm’s state. Next, for the “gen-action” stage, the KV cache is passed to the action expert, which produces a chunk of actions of size N_{chunk} through a diffusion process. An optional replan step then selects the first N_{replan} actions for execution to adjust performance efficiency trade-off. The “horizon” h is defined as the number of valid actions for each observation, which equals to N_{replan} . Finally, in “execution” stage, these h actions are executed sequentially on the robotic arm. After execution is completed, the observation is updated, and the next iteration begins.

In the aforementioned standard action chunking pipeline, each stage operates synchronously, meaning that every stage must wait for the preceding one to complete. As a result, between consecutive execution stages, there exists a halting gap of $T_{\text{halt}} = T_{\text{observe}} + T_{\text{gen-action}}$, based on actual profiling, the waiting time is substantial, comparable with the whole execution stage’s latency. It causes robot arm’s motion to become stuttered and inconsecutive. The fluency of execution can thus be characterized by T_{halt} , when T_{halt} approaches zero, the system achieves smooth and fluent execution. Similarly, the latency per action, which reflects the overall execution speed, can be expressed as the sum of the latencies of all stages: $T_{\text{action}} = (T_{\text{observe}} + T_{\text{gen-action}} + T_{\text{execution}})/h$ (abbreviated as T_o, T_g, T_e , respectively).

To achieve fast and fluent execution, we aim to minimize both T_{action} and T_{halt} . Existing model compression techniques [36, 38] can reduce latency by directly reducing the computational cost of T_o, T_g . However, it remains challenging to minimize T_{halt} to near zero value, since it inherently equals $T_o + T_g$. Instead of directly shortening individual stage latencies, we take a different approach—overlapping the latency of different stages to enable the VLA to operate in an **asynchronous and streaming** manner. We define O_{ge} and O_{oe} as the overlapped latency between the generation–execution and observation–execution stages, respectively. With these overlaps, the effective latency metrics can be expressed as:

$$\begin{aligned} T_{\text{action}} &= ((T_o + T_g + T_e) - (O_{ge} + O_{oe}))/h \\ T_{\text{halt}} &= (T_o + T_g) - (O_{ge} + O_{oe}). \end{aligned} \tag{1}$$

As shown above, the “streaming” optimization can work in parallel with model compression techniques that reduce T_g, T_o . It further overcomes the limitation of synchronous pipelines, and enables reducing T_{halt} toward nearly zero.

To achieve O_{ge} and O_{oe} , we develop techniques accordingly. As illustrated in the lower part of Fig. 2, we first adapt the streaming action flow matching to the VLA model, enabling it to efficiently generate actions one by one, allowing simultaneous action generation and execution. Secondly, we enhance the early observation mechanism so that the VLM can start processing in advance, before all action executions are completed, thereby achieving concurrent VLM processing and action execution.

4 Method

Based on the aforementioned analysis, we propose StreamingVLA, which enables parallelized execution along two dimensions. First, we introduce a new overlapping dimension between action generation and execution through state-based modeling for action flow matching (Sec. 4.1). This reformulates action modeling from predicting actions as absolute values to predicting actions as updates to a feature-space state that represents the physical system conditioned on prior actions. Second, to overlap action execution with the next iteration’s VLM observation processing, we introduce an improved early observation scheme (Sec. 4.2)

that exploits the diversity of action saliency and adaptively avoids early observation when the action saliency is high.

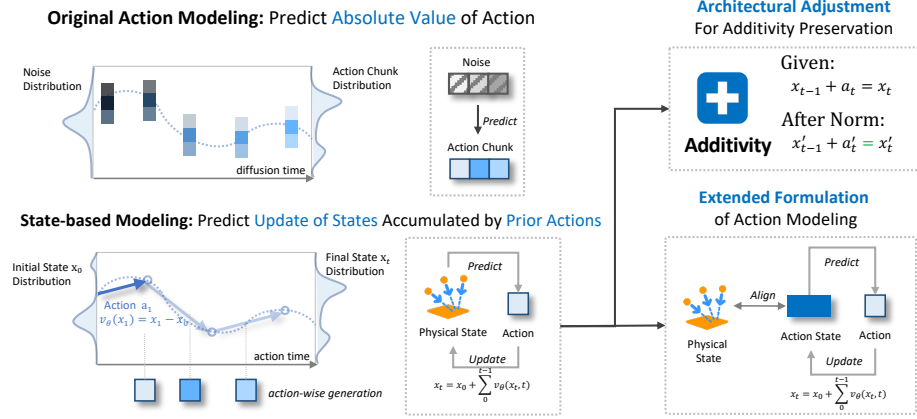


Fig. 3: The Illustration of state-based modeling of action flow matching: It reformulates the action modeling from predicting actions as absolute values to predicting actions as updates to a feature-space state accumulated by prior actions. Extended formulation and architectural adjustment is adopted for larger scaled VLA model and mainstream benchmarks.

4.1 State-based Modeling of Action Flow Matching

In this section, we first present the detailed formulation of action flow matching proposed in Streaming Flow Policy (SFP [19]). We then clarify the nontrivial challenge of adopting such scheme in larger-scaled VLA models and complex tasks. Then, we introduce our corresponding solution of extended formulation, and architectural adjustments.

Primary Formulation of Action Flow Matching In commonly adopted action chunking [25] scheme, The action expert of $\pi_{0.5}$ predicts an entire chunk of future actions based on the VLM processed observation o . For a noisy action chunk at a given timestep $t \in [0, 1]$, denoted as $\mathcal{A}_h^t$, where h implies the horizon (size of the chunk), the model predicts the velocity field $v_\theta(\mathcal{A}_h^t, t|o)$ along the flow trajectory from the noise distribution $p(\mathcal{A}_h^0)$ to the data distribution $p(\mathcal{A}_h^1)$. The model progressively denoise $\mathcal{A}_h^t$ until the final clean action chunk $\mathcal{A}_h^1$ is obtained. During this process, any intermediate result $\mathcal{A}_h^t$ contains neither complete nor accurate action information, thus only upon the completion of denoising process can all action tokens within the chunk be determined.

To enable parallelization between action generation and execution, action-wise generation, rather than chunk-wise generation, is required. A naive ap-

proach, equivalent to setting $N_{\text{chunk}} = 1$ in action chunking satisfies this requirement, but incurs excessive cost due to iterative denoising for each individual action, motivating the need for a more efficient formulation of action-wise generation. To address this, we adopt the action flow matching formulation proposed in Streaming Flow Policy (SFP [19]). Instead of denoising chunked actions, this formulation directly models the action trajectory using flow matching. It **reformulates action modeling from predicting absolute action values from a noise distribution, to predicting updates to the physical state conditioned on prior actions**. This formulation better aligns with the physical semantics of the task and enables efficient one-by-one action generation, facilitating parallelized execution between action generation and execution. Specifically, we train the model to learn the velocity field of the flow constructed by action trajectory. Given an action trajectory $\xi(t)$, we define its conditional action flow as follows:

$$v_\xi(x, t) = \dot{\xi}(t) - k(x - \xi(t)), \quad p_\xi^0(x) = \mathcal{N}(\xi(0), \sigma_0^2) \quad (2)$$

Where $p_\xi^0(x)$ denotes the initial distribution of the action states, modeled as a small Gaussian centered at $\xi(0)$ with variance σ_0^2 . The constant k serves as a stabilizing factor to constrain the behavior of flow when x does not strictly fall on trajectory $\xi(t)$. As demonstrated in SFP, the marginal distribution of x over time is:

$$p_\xi(x|t) = \mathcal{N}(\xi(t), \sigma_0^2 e^{-2kt}) \quad (3)$$

Indicating that at any time step t , the distribution of $x(t)$ forms a narrow Gaussian centered around $\xi(t)$, with variance exponentially decaying over time. It is also worth noting that the time step t here represents the normalized action index within horizon, instead of denoising timestep.

For training the action flow matching, from a dataset $p_{\mathcal{D}}$, we sample trajectories and their corresponding raw or processed observations $(\xi, o) \sim p_{\mathcal{D}}$. For each trajectory, we compute the flow velocity field $v_\xi(x, t)$ by 2, the model is trained by minimizing the conditional flow matching loss:

$$\begin{aligned} \mathcal{L}(v_\theta, p_{\mathcal{D}}) = \\ \mathbb{E}_{(o, \xi) \sim p_{\mathcal{D}}} \mathbb{E}_{t \sim U[0, 1]} \mathbb{E}_{x \sim p_\xi(x|t)} \|v_\theta(x, t|o) - v_\xi(x, t)\|_2^2. \end{aligned} \quad (4)$$

Once trained, the learned velocity field $v_\theta(x, t|o)$ can be integrated to obtain the state x_t along the conditional flow:

$$x_t = x_0 + \int_0^t v_\theta(x(\tau), \tau|o) d\tau. \quad (5)$$

The discretization of the integral gives:

$$x_T = x_0 + \sum_{t=0}^{T-1} v_\theta(x_t, t|o) \Delta t, \quad T \in \{1, 2, \dots, h\}. \quad (6)$$

The action we desired can be expressed as $a_t = x_{t+1} - x_t = v_\theta(x_t, t|o)\Delta t$. Therefore, each action expert forward produces a single velocity estimation $v_\theta(x_t, t|o)$, which can be immediately converted into one action token and executed, enabling streaming action generation.

Although the aforementioned formulation has been validated in relatively small-scale diffusion policy models and simpler tasks such as PushT [12], **extending it to mainstream benchmarks and large-scale VLA models remains non-trivial** for two main reasons:

1. The more complex control schemes in mainstream benchmarks require an extended action modeling formulation for effective adaptation.
2. The more complex architectures of VLA models must be carefully modified to preserve the essential additivity between actions.

To address these challenges, we propose the following corresponding solutions:

Extended Formulation of State-based Action Modeling: In the original formulation, the state x_t is identical to the robot’s physical state. This assumption holds for relatively simple tasks, where direct control logic can be applied. For example, in the PushT task, the policy directly outputs the physical space x and y coordinates for execution. However, in more complex mainstream benchmarks, the generated action is passed to an independent controller that indirectly affects the robot’s physical state. For instance, in the LIBERO benchmark, the Operational Space Controller (OSC_POSE) from robosuite converts takes in the policy output of end-effector pose deltas and calculates how it affects the whole simulation environment. Due to the nonlinearity introduced by the controller, the action can no longer directly represent the change in the physical state, rendering the original formulation inapplicable. To address this issue, we extend the formulation by introducing an **action-space state**, while ensuring its alignment with the corresponding **physical-space state**.

In practice, generated actions are used to update the action-space state, rather than the physical-space state. To fit such extended formulation and ensure alignment between action and physics space, the training scheme also should be modified accordingly. In the original training procedure, the flow matching process samples different sub-trajectories from the trajectory and performs prediction on these sub-trajectories.

While the absolute physical state is always accessible in the original formulation, in the extended formulation, the absolute action-space state, which is obtained by accumulating all preceding actions is not directly available during training. Since training only observes sub-trajectories, computing the action state by accumulating actions within merely a sub-trajectory would lead to a mismatch between the action-space state and the corresponding physical-space state, which may cause training failure. Therefore, before training, we precompute the complete action-space states along the trajectory, and input the initial action-space state corresponding to the physical state while training every sub-trajectory to ensure alignment.

In the inference stage, we maintain an action-space state x_t as part of model input, and obtain target velocity field $v_\theta(x_t, t)$ after a model forward pass. We take $a_t = v_\theta(x_t, t)\Delta t$ as the action output, and update the action-space state using:

$$x_{t+1} = x_t + v_\theta(x_t, t)\Delta t \quad (7)$$

thereby consistently preserving the alignment between the action-space state and the physical-space state.

Architectural Adjustment for Additivity Preservation: The key of state-based action modeling scheme is that it maintains a state that is updated through the cumulative of historical actions. The additivity between actions and states is an essential property. In small diffusion policy models, which composed of simple MLPs, naturally satisfy this property. However, in larger-scale VLA models, normalization layers introduced to stabilize training may break this additivity, which can lead to training collapse.

Specifically, the original normalization is defined as:

$$a'_t = (a_t - q_{\min})/\text{scale} \times 2 - 1, \quad \text{scale} = q_{\max} - q_{\min}, \quad (8)$$

where $q_{\max}$ and $q_{\min}$ are the maximum and minimum values of every dimension, which can be calibrated from the dataset in advance. When the original actions or states satisfy the additivity property $x_t + a_t = x_{t+1}$, the normalized variables generally fail to preserve this property, i.e., $x'_t + a'_t \neq x'_{t+1}$.

To preserve additivity, we introduce two modifications. First, we share the normalization statistics across all action-space states and actions so that the scaling factor remains consistent. Second, we modify the normalization layer by removing the offset term, resulting in:

$$a'_t = a_t/\text{scale}, \quad \text{scale} = q_{\max} - q_{\min}. \quad (9)$$

This simple modification is empirically effective in stabilizing training while preserving the additivity property.

4.2 Action Saliency-aware Adaptive Early Observation

The aforementioned action flow matching overlaps most of the action execution with action generation. To further reduce the halting time, we aim to overlap action execution with VLM processing. Early observation, which initiates VLM processing before the completion of the entire action sequence, is well suited for this purpose. Prior work such as RTC [5] and SmolVLA [33] achieves speedup by advancing observation processing. VLASH [35] further finetunes the model to mitigate the mismatch introduced by early observation. Despite the substantial speedups achieved, noticeable performance degradation still remains. In this work, we aim to further improve performance preservation under early observation.

Through the analysis of failure cases caused by improper early observation, we observe that "action saliency", which reflects the extent to which an action influences the resulting observation, varies significantly across actions. As

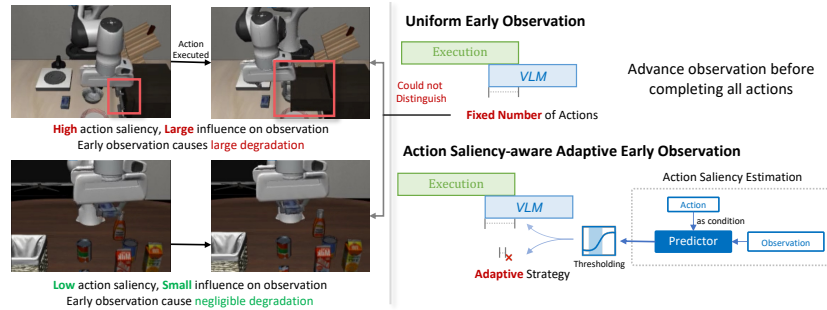


Fig. 4: The Illustration of action saliency aware adaptive early observation: We highlight the importance of diverse action saliency, and adopt a lightweight predictor to implement an adaptive early observation scheme.

illustrated in Fig. 4, the upper action completes pulling out the shelf, resulting in a significant environmental change. Performing early observation before this critical change leads to severe errors. In contrast, for most actions (such as the lower example), little observable environmental change occurs, and advance observation before executing these actions will not cause noticeable degradation. Consequently, a uniform early observation strategy that always skips the last few actions in a chunk may discard highly salient actions, leading to notable performance degradation. To address this issue, we propose an action-saliency-adaptive early observation scheme that avoids skipping actions with high saliency.

The key to accurate adaptive early observation is to properly measure action saliency in order to identify important actions. A straightforward approach is to use the norm of the predicted action tokens, as their physical interpretation suggests a positive correlation between token magnitude and movement in physical space. However, our empirical results show that although this strategy improves performance compared with naive uniform early observation, it still leads to noticeable degradation (See Tab. 1 for more details). To obtain a more accurate estimate of action saliency, we instead measure the influence of an action on the observation processing result. This feature representation directly affects future action generation, as it serves as the input to the action expert. We introduce a lightweight predictor to estimate this feature space difference.

Specifically, the lightweight transformer-based predictor takes the image embedding (extracted by the ViT in the VLM), which represents the current environment, as input. The remaining actions are fed into the predictor as conditional control signals, following the conditioning scheme used in DiT [30]. The predictor is trained to estimate the change in image embeddings after executing the remaining actions, using a simple mean squared error (MSE) loss. During inference, a thresholding strategy is applied to avoid performing early observation for actions with high saliency. We ensure that the additional overhead introduced by the predictor remains within an acceptable range. The runtime overhead, including the extra ViT processing, accounts for only about 5% of the model inference

time. Moreover, the training cost of the predictor is significantly lower than that of full fine-tuning of the entire model (e.g., VLASH). It is also worth noting that the lightweight predictor preserves flexibility and can readily augment existing techniques with adaptive capability.

Table 1: Main results on the LIBERO benchmark. We present the performance and efficiency of StreamingVLA and baseline methods. The T_{action} and T_{halt} are adopted to measure the speed and fluency of execution. The “AFM” and “AEO” represent the “action flow matching” and “adaptive early observation” technique. The “NEO” stands for the “naive early observation”. The “ANAO” stands for the “action norm based adaptive observation” as mentioned in Sec. 4.2

Method	Success Rate (%) $\uparrow$					Time per Action (ms) $\downarrow$ Halting Gap (ms) $\downarrow$	
	Spatial	Object	Goal	Long	Average	T_{action}	T_{halt}
$\pi_{0.5}$ ($h = 5$)	98.8	98.2	98.0	92.4	96.9	74.5	232.3
$\pi_{0.5}$ ($h = 10$)	97.4	98.2	96.2	88.6	95.1	49.9 (1.49 $\times$)	230.8 (1.01 $\times$)
RTC (d=1) [5]	96.2	19.8	93.0	25.2	58.55	50.2 (1.48 $\times$)	203.6 (1.14 $\times$)
SmolVLA [33]	97.0	99.0	97.0	90.0	95.8	51.1 (1.45 $\times$)	180.7 (1.28 $\times$)
VLASH [35]	97.5	99.2	97.3	94.6	97.1	40.6 (1.83 $\times$)	-
Temporal Ensembling	97.6	92.4	91.4	78.6	90.0	279.0 (0.26 $\times$)	231.6 (1.003 $\times$)
StreamingVLA (AFM)	97.6	99.0	98.8	93.0	97.1	33.7 (2.21 $\times$)	76.1 (3.05 $\times$)
StreamingVLA (AFM+NEO)	80.6	93.8	92.2	78.2	86.2	29.3 (2.54$\times$)	23.0 (10.10$\times$)
StreamingVLA (AFM+ANAO)	86.0	96.2	93.0	84.8	90.0	30.775 (2.42 $\times$)	27.75 (8.37 $\times$)
StreamingVLA (AFM+AEO)	96.6	96.6	95.4	91.0	94.9	31.625 (2.36 $\times$)	36.0 (6.45 $\times$)

5 Experiments

5.1 Implementation Details

Implementation Details: We adopt $\pi_{0.5}$ -Libero [17, 27] as our base model and conducted training on four NVIDIA A800-SXM4-80GB GPUs. Models are trained for 60,000 iterations with a learning rate of 1e-5, taking 10-20 GPU hours to finish. For the selection of the stabilizing factor k and the variance σ_0^2 defined in Sec. 4.1, we conduct grid search and select $k = 5$ and $\sigma_0^2 = 0.16$ for the subsequent training. Similarly, for the adaptive early observation predictor, we trained it for 54,000 iterations and employed a cosine learning rate schedule with an initial value of 1e-4. For the optimal threshold of the predictor, we evaluated both task success rate and inference speed across a range of values and ultimately selected the setting that achieves the best accuracy-latency trade-off. After training, models are evaluated on a single NVIDIA A800-SXM4-80GB GPU. All models are trained, deployed, and evaluated using the official PyTorch framework of OpenPI [13].

Task Suites: Following prior literature [5, 17, 33], We select four task suites from the Libero simulation environment: *Spatial*, *Object*, *Goal*, and *Long* to evaluate model performance. Each suite consists of 10 distinct tasks.

Baseline Methods: We adapt StreamingVLA to $\pi_{0.5}$ -Libero, a state-of-the-art VLA model featuring a flow-matching action expert. By default, $\pi_{0.5}$ -Libero uses $N_{\text{chunk}} = 10$ and replans to 5 actions, meaning only the first five actions from each generated chunk are executed, corresponding to a horizon size of $h = 5$. In action flow matching, actions are generated individually, so no replan is applied. For fair comparison, we also present results for $\pi_{0.5}$ without replan ($h = 10$), obtained using the official codebase. We also apply four baseline methods on the $\pi_{0.5}(h = 10)$ model, which also overlaps the latency of multiple stages:

- **Real-Time Chunking (RTC):** measure inference delay $d = \frac{T_o + T_g}{T_e/h}$ during execution, and use it as the early observation steps N_{eo} . The next chunk begins with these d actions already being executed, while the remaining $(h - d)$ actions are generated via an inpainting process, thereby enabling overlapped execution and generation. Following VLASH [35], we set a fix $d = 1$ in our evaluation.
- **SmolVLA:** perform an early observation before the current chunk finishes execution. The executed actions are removed from the new chunk.
- **VLASH:** finetune the model with rectified states to mitigate misalignment caused by early observation.
- **Temporal Ensembling:** maintain a buffer of predicted action chunks and executing, at each timestep, the average of all actions predicted for that timestep.

Evaluation Metrics: We perform 50 rollouts per task across all four suites, and evaluate the following metrics:

- **Success Rate (%)**: The percentage of successful rollouts among all attempts.
- **Timing per action T_{action} (ms)**: The average time per executed action, computed as total time divided by horizon.
- **Halting time T_{halt} (ms)**: The average idle time between executions, measuring the fluency.

5.2 Experimental Results

The comparison of efficiency-accuracy trade-off with baseline methods is presented in Tab. 1, We conclude the key findings as follows:

(1) StreamingVLA effectively reduces latency & halting Gap, achieving fast and fluent execution. StreamingVLA achieves an average success rate of 94.9%, compared with 95.1% for $\pi_{0.5}$ ($h=10$), indicating negligible performance degradation of less than 0.5%. In terms of overall latency, StreamingVLA yields a $1.57\times$ speedup over $\pi_{0.5}$ ($h=10$) and $2.35\times$ over $\pi_{0.5}$ ($\text{replan}=5$). Moreover, the Halting time is significantly reduced from around 232.3 ms to 36.0ms (a $6.45\times$ improvement), demonstrating substantial improvement on execution fluency. It is also worth noting that when adopting action flow matching (AFM) alone, the model improves success rates across all suites, while achieving $2.21\times$

and $1.48\times$ latency reductions and a $3.05\times$ halting gap improvement compared to $\pi_{0.5}$ ($h=5/10$).

(2) Baseline methods face challenges and limitations for speedup.

For the RTC baseline, the primary speed gain comes from a shorter halting gap (203.6 ms, $1.14\times$), which reduces the waiting time between observations and action execution. However, this early observation strategy leads to a pronounced drop in success rate, with the average performance falling to 58.55% and long-horizon tasks degrading drastically. Although RTC attempts to refine the predicted velocity field through gradient-based correction during inference, the improvement is limited and insufficient to compensate for the information loss caused by premature observation. As seen in Tab. 1, SmolVLA could moderately reduce halting from 230 to 180 ms, and the latency remain nearly unimproved (51.1 vs. 49.9). VLASH and our method optimize early observation from distinct dimensions. While training-based VLASH maintains success rates under acceleration, StreamingVLA achieves superior speed–performance trade-off by enabling parallel action generation and execution. The naive early observation (NEO), could significantly reduce latency and halting, but incur severe performance degradation (from 97.1 to 86.2), our adaptive early observation, could preserve the performance (94.7) while delivering substantial speedup.

Table 2: Ablation studies for StreamingVLA techniques validate the effectiveness of our Action Flow Matching (AFM) formulation w/o state-based alignment modeling (SA), normalization modification (NM) mentioned in Sec. 4.1, as well as adaptive early observation (EO) schemes. The “2” and “1.45” represents the average number of early observed actions. The "Random" represents employing early observation randomly.

AFM	NM	SA	Horizon	EO	Success Rate(%)	T_{action} (ms)	T_{halt} (ms)
-	-	-	10	-	95.1	49.9	230.78
✓	-	-	10	-	-	-	-
✓	✓	-	10	-	61.8	-	-
✓	✓	✓	10	-	97.1	33.7	76.1
✓	✓	✓	10	Naive (2)	86.2	29.3	23.0
✓	✓	✓	10	Random (1.4)	90.875	32.125	38.3
✓	✓	✓	10	Adaptive (1.4)	94.9	31.625	36.0

6 Ablation Studies

To validate the effectiveness of the proposed techniques, we present the results in Tab. 2. The key findings are summarized as follows:

(1) Effectiveness of Action Flow Matching (AFM). Replacing the vanilla formulation with AFM significantly improves temporal modeling of actions. Although AFM alone does not directly report full metrics in the table, its combination with subsequent components consistently yields better stability and performance, indicating that modeling action evolution as a continuous flow provides a more structured optimization objective.

(2) Critical role of Normalization Modification (NM). Normalization Modification is a key component of the new AFM formulation. Without NM, the model fails to train effectively under the action flow parameterization. As shown in the table, introducing AFM without proper normalization does not yield stable or competitive results, while enabling NM establishes a well-conditioned training objective. Although NM alone (without SA) leads to poor final performance (61.8% success rate), this does not indicate ineffectiveness; rather, it shows that NM is a necessary but not sufficient component. In other words, NM provides the essential scaling and stabilization required for optimizing the continuous action flow, and meaningful performance can only be achieved when NM is combined with state-based alignment.

(3) Importance of State-based Alignment (SA). Adding SA on top of AFM and NM dramatically improves the success rate to 97.1%, surpassing the baseline (95.1%), while simultaneously reducing both action inference time (T_{action} : 49.9 $\rightarrow$ 33.7 ms) and halting time (T_{halt} : 230.78 $\rightarrow$ 76.1 ms). This verifies that maintaining consistency between action-space state and physical state is crucial for both accuracy and efficiency.

(4) Early Observation (EO) for latency reduction. Naive EO aggressively exposes two future actions at each step, achieving the lowest latency but causing a clear performance drop (86.2%), indicating that excessive early observation harms decision quality.

Random and Adaptive EO share the same average number of early observed actions (1.4), ensuring a fair comparison. While Random EO provides a reasonable speed–performance trade-off, Adaptive EO further improves success rate (94.9% vs. 90.9%) by leveraging an adaptor to guide when early observation should be triggered, demonstrating that informed scheduling is more effective than unguided randomness.

Overall, the ablation study confirms that (1) AFM provides a principled action modeling framework, (2) SA is essential for stabilizing AFM-based training, and (3) Adaptive Early Observation effectively reduces latency while largely preserving task success, validating the design of StreamingVLA.

7 Conclusions

To address the stuttered and halting execution issue of VLA, we present StreamingVLA, which enables parallel processing of multiple stages in VLA in a streaming manner, without sacrificing performance. It achieves a $2.4\times$ speedup in latency and a $6.5\times$ reduction in halting time, enabling fast and fluent execution.

References

1. Gr-3 technical report. Tech. rep., ByteDance-Seed (Jul 2025), <https://seed.bytedance.com/GR3>, technical Report, Project page: <https://seed.bytedance.com/GR3>
2. Bharadhwaj, H., Vakil, J., Sharma, M., Gupta, A., Tulsiani, S., Kumar, V.: Roboagent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). pp. 4788–4795 (2024). <https://doi.org/10.1109/ICRA57147.2024.10611293>
3. Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al.: Gr00t v1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734 (2025)
4. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L.X., Tanner, J., Vuong, Q., Walling, A., Wang, H., Zhilinsky, U.: π_0 : A vision-language-action flow model for general robot control (2024), <https://arxiv.org/abs/2410.24164>
5. Black, K., Galliker, M.Y., Levine, S.: Real-time execution of action chunking flow policies. arXiv preprint arXiv:2506.07339 (2025)
6. Black, K., Ren, A.Z., Equi, M., Levine, S.: Training-time action conditioning for efficient real-time chunking (2025), <https://arxiv.org/abs/2512.05964>
7. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., Florence, P., Fu, C., Arenas, M.G., Gopalakrishnan, K., Han, K., Hausman, K., Herzog, A., Hsu, J., Ichter, B., Irpan, A., Joshi, N., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, L., Lee, T.W.E., Levine, S., Lu, Y., Michalewski, H., Mordatch, I., Pertsch, K., Rao, K., Reymann, K., Ryoo, M., Salazar, G., Sanketi, P., Sermanet, P., Singh, J., Singh, A., Soricut, R., Tran, H., Vanhoucke, V., Vuong, Q., Wahid, A., Welker, S., Wohlhart, P., Wu, J., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., Zitkovich, B.: Rt-2: Vision-language-action models transfer web knowledge to robotic control (2023), <https://arxiv.org/abs/2307.15818>
8. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jackson, T., Jesmonth, S., Joshi, N.J., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, K.H., Levine, S., Lu, Y., Malla, U., Manjunath, D., Mordatch, I., Nachum, O., Parada, C., Peralta, J., Perez, E., Pertsch, K., Quiambao, J., Rao, K., Ryoo, M., Salazar, G., Sanketi, P., Sayed, K., Singh, J., Sontakke, S., Stone, A., Tan, C., Tran, H., Vanhoucke, V., Vega, S., Vuong, Q., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., Zitkovich, B.: Rt-1: Robotics transformer for real-world control at scale (2023), <https://arxiv.org/abs/2212.06817>
9. Cheang, C.L., Chen, G., Jing, Y., Kong, T., Li, H., Li, Y., Liu, Y., Wu, H., Xu, J., Yang, Y., et al.: Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. arXiv preprint arXiv:2410.06158 (2024)
10. Chen, Y., Li, X.: Rlrc: Reinforcement learning-based recovery for compressed vision-language-action models. arXiv preprint arXiv:2506.17639 (2025)
11. Cheng, A.C., Ji, Y., Yang, Z., Gongye, Z., Zou, X., Kautz, J., Biyik, E., Yin, H., Liu, S., Wang, X.: Navila: Legged robot vision-language-action model for navigation. arXiv preprint arXiv:2412.04453 (2024)

12. Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., Song, S.: Diffusion policy: Visuomotor policy learning via action diffusion (2024), <https://arxiv.org/abs/2303.04137>
13. Contributors, O.: Open-pi: An open-source library for vision-language-action models. <https://github.com/open-pi/open-pi> (2024), accessed: 2026-03-06
14. Cui, C., Ding, P., Song, W., Bai, S., Tong, X., Ge, Z., Suo, R., Zhou, W., Liu, Y., Jia, B., Zhao, H., Huang, S., Wang, D.: Openhelix: A short survey, empirical analysis, and open-source dual-system vla model for robotic manipulation (2025), <https://arxiv.org/abs/2505.03912>
15. George, A., Farimani, A.B.: One act play: Single demonstration behavior cloning with action chunking transformers. arXiv preprint arXiv:2309.10175 (2023)
16. Guan, W., Hu, Q., Li, A., Cheng, J.: Efficient vision-language-action models for embodied manipulation: A systematic survey (2025), <https://arxiv.org/abs/2510.17111>
17. Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al.: $\pi 0$. 5: a vision-language-action model with open-world generalization, 2025. URL <https://arxiv.org/abs/2504.16054> 1(2), 3
18. Jeong, H., Lee, H., Kim, C., Shin, S.: A survey of robot intelligence with large language models. Applied Sciences **14**(19) (2024). <https://doi.org/10.3390/app14198868>, <https://www.mdpi.com/2076-3417/14/19/8868>
19. Jiang, S., Fang, X., Roy, N., Lozano-Pérez, T., Kaelbling, L.P., Ancha, S.: Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories (2025), <https://arxiv.org/abs/2505.21851>
20. Jiang, Y., Cheng, S., Ding, Y., Gao, F., Qi, B.: Asyncvla: Asynchronous flow matching for vision-language-action models (2025), <https://arxiv.org/abs/2511.14148>
21. Kim, M.J., Finn, C., Liang, P.: Fine-tuning vision-language-action models: Optimizing speed and success (2025), <https://arxiv.org/abs/2502.19645>
22. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., Finn, C.: Openvla: An open-source vision-language-action model (2024), <https://arxiv.org/abs/2406.09246>
23. Kim, Y., Kim, D., Choi, J., Park, J., Oh, N., Park, D.: A survey on integration of large language models with intelligent robots. Intelligent Service Robotics **17**(5), 1091–1107 (Aug 2024). <https://doi.org/10.1007/s11370-024-00550-5>, <http://dx.doi.org/10.1007/s11370-024-00550-5>
24. Lai, L., Huang, A.Z., Gershman, S.J.: Action chunking as policy compression. PsyArXiv (2022)
25. Li, Q., Zhou, Z., Levine, S.: Reinforcement learning with action chunking. arXiv preprint arXiv:2507.07969 (2025)
26. Li, Z., Wu, X., Du, H., Liu, F., Nghiem, H., Shi, G.: A survey of state of the art large vision language models: Alignment, benchmark, evaluations and challenges (2025), <https://arxiv.org/abs/2501.02189>
27. Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., Stone, P.: Libero: Benchmarking knowledge transfer for lifelong robot learning. Advances in Neural Information Processing Systems **36**, 44776–44791 (2023)
28. Ma, Y., Song, Z., Zhuang, Y., Hao, J., King, I.: A survey on vision-language-action models for embodied ai (2025), <https://arxiv.org/abs/2405.14093>

29. O'Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlekar, A., Jain, A., et al.: Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In: 2024 IEEE International Conference on Robotics and Automation (ICRA). pp. 6892–6903. IEEE (2024)
30. Peebles, W., Xie, S.: Scalable diffusion models with transformers (2023), <https://arxiv.org/abs/2212.09748>
31. Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., Levine, S.: Fast: Efficient action tokenization for vision-language-action models. arXiv preprint arXiv:2501.09747 (2025)
32. Sapkota, R., Cao, Y., Roumeliotis, K.I., Karkee, M.: Vision-language-action models: Concepts, progress, applications and challenges (2025), <https://arxiv.org/abs/2505.04769>
33. Shukor, M., Aubakirova, D., Capuano, F., Kooijmans, P., Palma, S., Zouitine, A., Aractingi, M., Pascal, C., Russi, M., Marafioti, A., Alibert, S., Cord, M., Wolf, T., Cadene, R.: Smolvla: A vision-language-action model for affordable and efficient robotics (2025), <https://arxiv.org/abs/2506.01844>
34. Song, W., Chen, J., Ding, P., Zhao, H., Zhao, W., Zhong, Z., Ge, Z., Ma, J., Li, H.: Accelerating vision-language-action model integrated with action chunking via parallel decoding. arXiv preprint arXiv:2503.02310 (2025)
35. Tang, J., Sun, Y., Zhao, Y., Yang, S., Lin, Y., Zhang, Z., Hou, J., Lu, Y., Liu, Z., Han, S.: Vlash: Real-time vlas via future-state-aware asynchronous inference (2025), <https://arxiv.org/abs/2512.01031>
36. Wen, J., Zhu, Y., Li, J., Zhu, M., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., Peng, Y., Feng, F., Tang, J.: Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. arXiv preprint arXiv:2409.12514 (2024)
37. Xu, S., Wang, Y., Xia, C., Zhu, D., Huang, T., Xu, C.: Vla-cache: Efficient vision-language-action manipulation via adaptive token caching (2025), <https://arxiv.org/abs/2502.02175>
38. Yang, Y., Wang, Y., Wen, Z., Zhongwei, L., Zou, C., Zhang, Z., Wen, C., Zhang, L.: Efficientvla: Training-free acceleration and compression for vision-language-action models. arXiv preprint arXiv:2506.10100 (2025)
39. Ye, W., Wu, Q., Lin, W., Zhou, Y.: Fit and prune: Fast and training-free visual token pruning for multi-modal large language models. arXiv preprint arXiv:2409.10197 (2024)
40. Zhang, R., Dong, M., Zhang, Y., Heng, L., Chi, X., Dai, G., Du, L., Du, Y., Zhang, S.: Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. arXiv preprint arXiv:2503.20384 (2025)
41. Zhao, T.Z., Kumar, V., Levine, S., Finn, C.: Learning fine-grained bimanual manipulation with low-cost hardware (2023), <https://arxiv.org/abs/2304.13705>
42. Zhen, H., Qiu, X., Chen, P., Yang, J., Yan, X., Du, Y., Hong, Y., Gan, C.: 3d-vla: A 3d vision-language-action generative world model. arXiv preprint arXiv:2403.09631 (2024)
43. Zheng, R., Liang, Y., Huang, S., Gao, J., Daumé III, H., Kolobov, A., Huang, F., Yang, J.: Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. arXiv preprint arXiv:2412.10345 (2024)

Appendix

A Real-World Robot Deployment

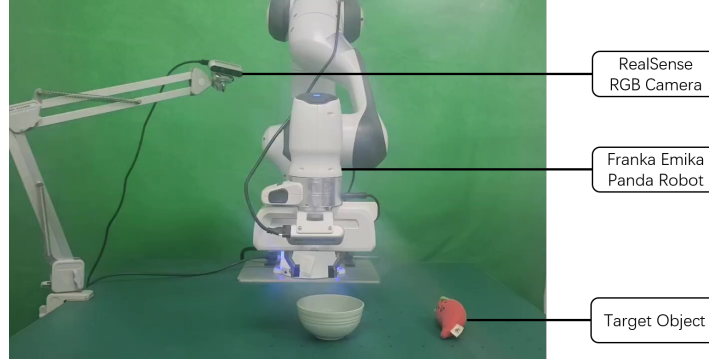


Fig. 5: Real-world Setup. The real-world evaluation platform includes a tabletop workspace, a Franka Panda robotic manipulator fixed to the table, and a RGB camera for visual observation. All objects are positioned on the table surface.

To validate the practical efficiency of our method, we deploy the model on a real-world robotic manipulation task. The task requires a robotic arm to pick up an object from a designated location and place it at a target position. This pick-and-place scenario represents a common manipulation setup that requires precise action execution and timely perception updates.

Our model is trained based on the $\pi_{0.5}$ -**base** architecture with an action horizon of $H = 8$. During deployment, we compare our **StreamingVLA** framework with the original $\pi_{0.5}$ policy under the same hardware setup. For the baseline configuration, the $\pi_{0.5}$ policy operates with an action horizon of $H = 8$ and a replanning step of 4 actions, while **StreamingVLA** uses the same horizon but performs streaming action generation.

Both methods are deployed on the same robotic platform as Fig. 5 to ensure a fair comparison. The system consists of a table-top workspace with a Franka Emika Panda robot and a fixed RGB camera, whose captured images serve as visual input to the VLA model. The robot has seven actuated joints and a parallel-jaw gripper with open/close capability. We measure the average time per action demonstrated in Sec. 5 during the manipulation process. The results show that **StreamingVLA** achieves an average latency of **170.88 ms** per action, while the original $\pi_{0.5}$ baseline requires **271.49 ms** per action.

These results demonstrate that **StreamingVLA** significantly improves real-world control efficiency by reducing action generation latency while maintaining the same action horizon. The reduced latency enables faster response during execution, which is critical for real-time robotic manipulation.

Algorithm 1 Training algorithm

Require: Preprocessed training set $\mathcal{D} = \{(o_i, \alpha_i, \xi_i)\}_{i=1}^N$, horizon h , noise scale σ_0 , stabilizing factor k

- 1: **while** not converged **do**
- 2: $(o_i, \alpha_i, \xi_i) \sim \mathcal{D}$
- 3: Compute action state $\mathcal{A}_i$:
- 4: $\mathcal{A}_i[n] \leftarrow \mathcal{A}_i[n-1] + \xi_i[n-1]$, $\mathcal{A}_i[0] \leftarrow \alpha_i$
- 5: $t \sim \text{Uniform}(0, 1)$, index $T \leftarrow \lfloor t \cdot h \rfloor$
- 6: $x_T \sim \mathcal{N}(\mathcal{A}_i[T], \sigma_0^2 e^{-2kT/h})$
- 7: $\theta \leftarrow \theta - \lambda \nabla_{\theta} \underbrace{\|v_{\xi}(x_T, T/h) - v_{\theta}(x_T, T/h|o_i)\|^2}_{\text{Conditional flow matching loss}}$
- 8: **end while**
- 9: **return** v_{θ}

B Algorithms

We introduce the action-space state α as a new type of input during training. These states are directly read from the preprocessed dataset and used as the initial action state α_i of each sub-trajectory ξ_i , from which the ground-truth action states of the entire sub-trajectory $\mathcal{A}_i$ are computed. Consequently, our training algorithm differs from that of the streaming flow policy or $\pi_{0.5}$; We present the detailed description in Algorithm 1 as follows:

During inference we also need to maintain the action state, which is updated in real time according to the actions that have been executed. The corresponding procedure is described in Algorithm 2.

C Additional Experimental Results

C.1 Runtime Breakdown

We further analyze the runtime composition based on actual profiling using the analysis framework discussed in Sec. 3 and illustrated in Fig. 6, to evaluate the baseline method and identify the sources of the speedup in StreamingVLA.

For $\pi_{0.5}$, no acceleration methods are applied thus $O_{ge} = O_{oe} = 0$, the latency and halting time could be calculated as follows, matching the measured latencies:

$$T_{\text{action1}} = \frac{T_o + T_g + h_1 T_e}{h_1} = 74.6\text{ms},$$

$$T_{\text{halt1}} = T_o + T_g = 238\text{ms}.$$

Algorithm 2 Inference algorithm

Require: $v_\theta(x_T, T/h|o)$, Early observation step N_{eo} , Observation indicator I with threshold η

```

1:  $o \leftarrow o_0, \alpha \leftarrow \mathbf{0}, T \leftarrow 0, \text{need\_obs} \leftarrow \text{True}$  //initialization
2: while True do
3:   if need_obs then
4:      $o \leftarrow \text{Observation}$  //async process observation
5:     need_obs  $\leftarrow$  False
6:   end if
7:   action  $a \leftarrow v_\theta(\alpha, T/h|o)^{\frac{1}{h}}$  //async generate action
8:   if  $h - T = N_{oe}$  then
9:     if  $I(a) \leq \eta$  then //adaptive early observation
10:      need_obs  $\leftarrow$  True
11:    end if
12:  end if
13:  if  $h - T = 0$  then
14:    need_obs  $\leftarrow$  True
15:     $T \leftarrow 0$ 
16:  end if
17:  execute( $a$ ) //async execute action
18:   $\alpha \leftarrow \alpha + a$  //update action state
19:   $T \leftarrow T + 1$ 
20: end while

```

For **StreamingVLA**, The overlap time O_{ge} and O_{oe} can be calculated as below:

$$O_{ge} = (h - 1) \min \left\{ \frac{T_g}{h}, \frac{T_e}{h} \right\} = 162\text{ms},$$

$$O_{oe} = N_{eo} \frac{T_e}{h} = 41.58\text{ms}.$$

Then we obtain:

$$T_{\text{action2}} = \frac{T_o + T_g + T_e - O_{ge} - O_{oe}}{h} = 30.44\text{ms},$$

$$T_{\text{halt2}} = T_o + T_g - (O_{ge} + O_{oe}) = 34.42\text{ms}.$$

In summary, StreamingVLA delivers a $2.40\times$ latency and $6.71\times$ halting gap improvements.

C.2 Design Details of Early Observation Predictor

The architecture of our predictor model is illustrated in Fig. 7. We utilized two NVIDIA A800-SXM4-80GB GPUs to train the predictor, taking 3-4 GPU hours to finish. During training, we randomly sample frame pairs with a temporal gap of 1-3 actions. A visual encoder first maps these observations into an embedding space. The predictor then takes the early frame embedding as input

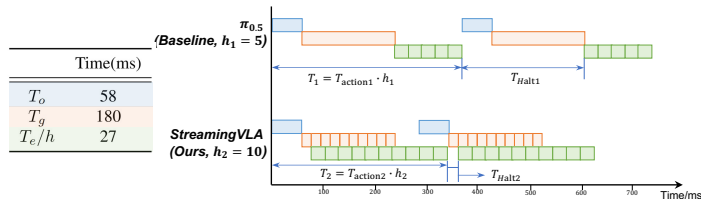


Fig. 6: The runtime breakdown based on actual profiling of $\pi_{0.5}$ and StreamingVLA. The three colored blocks correspond to the “observation”, “gen-action” and “execution” stage in the VLA workflow. The length of each block reflects the measured time.

and the pending action sequence as condition to generate a residual embedding, $\Delta embedding$. The model is optimized using a Mean Squared Error (MSE) loss between the latent representation of the late frame and the predicted composite ($embedding_{early} + \Delta embedding$).

During inference, the L_2 -norm of the predicted $\Delta embedding$ serves as the metric for visual variance. Early observation is inhibited if this value exceeds a predefined threshold, and we select this threshold by analyzing the accuracy-latency tradeoff across various settings.

In our framework, each observation generates a horizon of H actions ($H = 10$ in our setting), which are appended to the pending execution queue. When the queue reduces to n remaining actions (a predefined parameter), the current observation and residual actions are transmitted to the server. The predictor in the server then evaluates whether to trigger an early observation.

Consequently, the predictor operates once per H actions, introducing a marginal latency of 8–10 ms per invocation. This translates to an average computational overhead of approximately 0.8–1 ms per action, representing only 5% of the standard generation time (18 ms). Despite this minimal overhead, our method achieves a significant accuracy improvement of 8.7% compared to the naive early observation baseline.

C.3 Visualization

We visualized representative tasks from each suite to provide an intuitive comparison of task completion speed and fluency between our method and the baselines. For each suite, we selected one representative task for visualization. The tasks corresponding to the four suites are as follows:

- **LIBERO-Spatial:** pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate.
- **LIBERO-Object:** pick up the orange juice and place it in the basket.
- **LIBERO-Goal:** open the top drawer and put the bowl inside.
- **LIBERO-Long:** put both the alphabet soup and the tomato sauce in the basket.

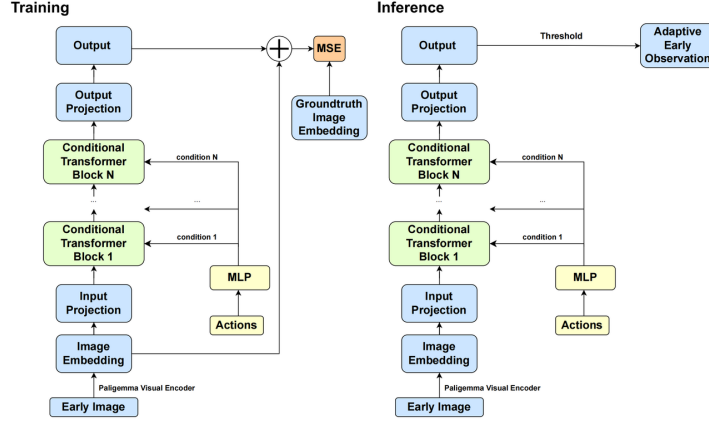


Fig. 7: Architecture of the predictor for adaptive early observation. During training, the model receives the early-frame embedding and the pending action sequence as conditions and predicts a residual embedding $\Delta embedding$ that estimates the latent change toward the future frame. The model is trained using an MSE loss between the ground-truth late-frame embedding and the reconstructed representation ($embedding_{early} + \Delta embedding$). At inference time, the L_2 norm of $\Delta embedding$ serves as a visual-variance indicator, which is compared with a threshold to determine whether early observation should be triggered.

We recorded the simulation state every 30 ms (approximately the time required to execute one action step in the LIBERO environment), and used these snapshots to generate video results. A subset of these results is shown in Fig. 8. During the “open the top drawer and put the bowl inside” task, the robotic arm controlled by the $\pi_{0.5}$ model experienced 12 pauses while opening the drawer, and even failed to open it once due to a prolonged pause. When picking up the bowl and placing it inside the drawer, the arm encountered an additional 25 pauses. These interruptions significantly delayed the task completion and even hindered its success. In contrast, the robotic arm controlled by StreamingVLA exhibited no noticeable halting throughout the execution of this task.

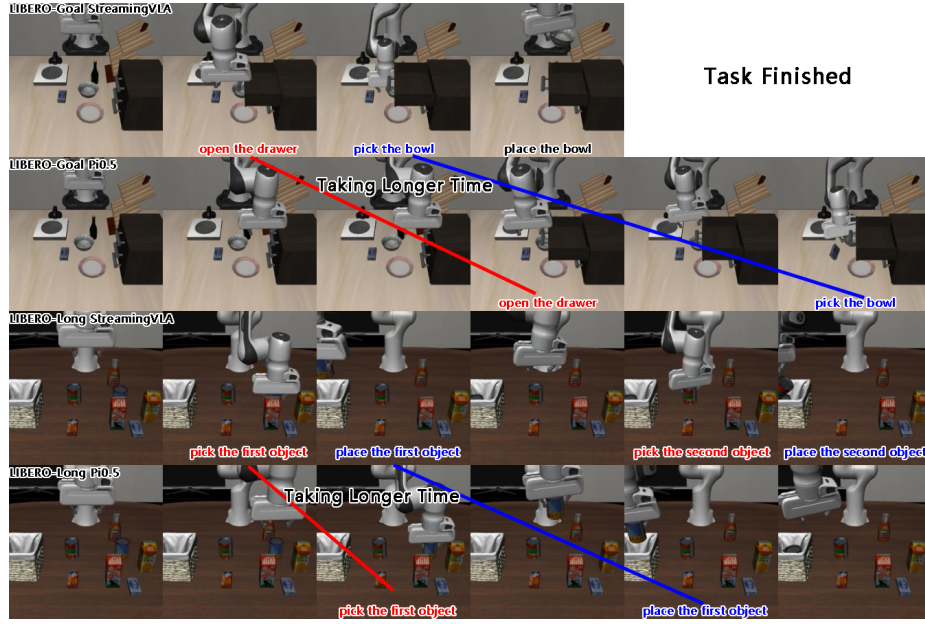


Fig. 8: Screenshots of our video simulation results. The images in each row are spaced 4 seconds apart. **Top two rows:** *open the top drawer and put the bowl inside.* StreamingVLA completes the task in under 16 seconds, whereas pi0.5 has not yet picked up the bowl at the 20-second mark. **Bottom two rows:** *put both the alphabet soup and the tomato sauce in the basket.* At 20 seconds, StreamingVLA has already finished the pick-and-place operations for both objects, while pi0.5 is still on its way to picking up the second item.